Campus Event Management & Feedback Analyzer

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410049	26102000171	BWU/MCA/26/054	Jyotirmoy Das

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ABSTRACT

This project presents a web-based application for managing university events, built with Python and Streamlit. The system lets administrators create events with a fixed participant capacity, lets students register for them, and keeps a record of every registered participant. After an event, registered students can submit a star rating (1 to 5) and an optional comment.

The application validates all input before saving it: event dates in the past are rejected, e-mail addresses are checked against a strict format, duplicate registrations are blocked, and registration closes automatically when an event is full. Data is stored in lightweight JSON files, so no database server is required. Collected feedback is analysed with pandas and visualised with matplotlib through average-rating, rating-distribution and participation charts. The project shows how a small, modular Python application can replace manual spreadsheets for campus event administration and give organisers measurable insight into event quality.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	1
2	Objective	2
3	Planning	3
4	Requirement Analysis	4-5
5	System Flow	6-10
6	Proposed Design	11-13
7	Experimental Result	14-16
8	Future Scope	17
9	Conclusion	18
10	References	19

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	Context Level Diagram	6
5.2	Data Flow Diagram Level 0	6
5.3	Data Flow Diagram Level 1	7
5.4	Use Case Diagram	7
5.5	Activity Diagram	8
5.6	Sequence Diagram	9
5.7	Flow Chart	10
7.1	View All Events Screen	14
7.2	Create Event Screen	14
7.3	Register Student Screen	14
7.4	Registered Students and Participation Summary	15
7.5	Feedback Analysis Output	15
7.6	Generated Charts	16

LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE

Symbol / Abbreviation	Description
CSV	Comma Separated Values (file format used for event and registration data input)
DFD	Data Flow Diagram
CLD	Context Level Diagram
UML	Unified Modeling Language
GUI	Graphical User Interface (Streamlit web interface)
Avg	Average (used for average ratings and attendance)
ID	Identifier (Event ID / Student ID)
	Data Flow / Message Direction (in DFD and Sequence Diagram)
■/ ■	Process / Activity Symbol (in Flow Chart and Activity Diagram)
■	Decision Symbol (in Flow Chart and Activity Diagram)
■(open)	Data Store (in DFD)
Actor	External Entity / User (in Use Case and Context Diagrams)

CHAPTER 1: INTRODUCTION

Universities organise seminars, workshops, cultural programmes and technical competitions throughout the year. In many departments these events are still handled with paper forms, messaging groups or loose spreadsheets. This leads to common problems: duplicate sign-ups, events that accept more people than the venue can hold, missing contact details, and almost no structured feedback once the event is over.

This project, the **Campus Event Management & Feedback Analyzer**, addresses these problems with a single Python application. Organisers can create events, students can be registered against them, registrations can be searched or cancelled, and feedback can be collected and analysed. The user interface is built with Streamlit, so the whole system runs in a web browser with no front-end coding.

The application follows a simple, modular structure. Plain Python classes (`Event`, `Student`, `Registration`) define the data, a manager class (`CampusEventManager`) handles loading and saving, and custom exceptions describe business-rule failures clearly. Data is persisted in JSON files, and pandas and matplotlib turn the stored records into tables and charts. The system is intentionally lightweight and easy to understand, which makes it suitable for students who want to learn how a complete data-driven application is put together.

CHAPTER 2: OBJECTIVE

The objective of this project is to develop a Python application that manages university events from creation to feedback analysis. Specifically, the system aims to:

1. Allow organisers to create events with an ID, name, date, venue and maximum number of participants.
2. Allow students to register for events while enforcing capacity limits and preventing duplicate registrations.
3. Maintain participant information (student ID, name, e-mail and registration time) and let users view the list of registered students for any event.
4. Allow registrations to be cancelled.
5. Collect post-event feedback (rating and comment) from registered students, accepting only one response per student per event.
6. Analyse the feedback to compute overall and event-wise average ratings, and present participation and rating data as charts.
7. Validate all user input and report errors with clear messages instead of crashing.

CHAPTER 3: PLANNING

The planning phase set out the step-by-step approach used to design and implement the system, so that the application stays modular and easy to maintain.

1. Requirement Identification

- Understand how campus events are currently managed and where the process breaks down.
- Identify the users: event organisers/administrators and student participants.
- Define the core requirement: create events → register students → collect feedback → analyse results.

2. System Design Planning

- Use Python as the implementation language and Streamlit for the web interface.
- Use JSON files for storage so the project needs no database installation.
- Use pandas for tabular display and aggregation, and matplotlib for charts.
- Define separate classes for events, students and registrations, and a manager class for persistence.

3. Module Structure

The application is divided into nine functional modules, accessible from a sidebar menu: View All Events, Create Event, Register Student, Search Registered Students, Cancel Registration, Collect Feedback, Participation Summary, Analyze Feedback and Generate Charts.

4. Data Storage Planning

Four JSON files are stored in a folder named `campus_event_data`: `events.json`, `students.json`, `registrations.json` and `feedback.json`. Each file holds a list of records, and the files are created automatically on first use.

5. Validation Planning

Business rules were listed before coding: no past event dates, unique event IDs, no empty fields, valid e-mail format, no duplicate registration, no registration beyond capacity, and feedback only from registered students, once per event.

6. Testing & Validation Plan

- Test each module with valid input, then with invalid and boundary input (full event, duplicate entries, malformed e-mail, past date).
- Verify that data survives an application restart.
- Verify that tables and charts reflect the stored data correctly.

7. Scalability & Improvement Planning

Plan for later additions such as authentication, a database back end, e-mail notifications and exportable reports.

CHAPTER 4: REQUIREMENT ANALYSIS

1. Functional Requirements

No.	Requirement
FR1	The system shall display all events with their ID, name, date, venue and participant count (registered/maximum).
FR2	The system shall create an event only if the ID is unique, the name and venue are non-empty, and the date is today or in the future.
FR3	The system shall register a student for an event after validating the ID, name and e-mail.
FR4	The system shall reject duplicate registrations of the same student for the same event.
FR5	The system shall reject registration when the number of registered students has reached the event's maximum.
FR6	The system shall list the registered students of a selected event with name, e-mail and registration time.
FR7	The system shall cancel a registration identified by event ID and student ID.
FR8	The system shall accept a 1–5 rating and an optional comment only from a currently registered student, once per event.
FR9	The system shall show a participation summary for all events.
FR10	The system shall compute the total number of responses, the overall average rating and the average rating per event.
FR11	The system shall generate charts for average rating per event, distribution of ratings and participation per event.

2. Non-Functional Requirements

- **Usability:** a simple sidebar-driven web interface that requires no technical knowledge.
- **Reliability:** missing or corrupted data files must not crash the application; they are treated as empty.
- **Data integrity:** all changes are written to disk immediately after each successful operation.
- **Portability:** runs on any system with Python and the required libraries.
- **Maintainability:** separate classes and exceptions keep logic readable and extendable.

3. Software Requirements

Component	Details
Language	Python 3
Web framework	Streamlit
Data analysis	pandas
Visualisation	matplotlib
Standard libraries	json, os, re, datetime
Storage	JSON files
Browser	Any modern web browser

4. Data Requirements

File	Fields
events.json	event_id, name, date, venue, max_participants
students.json	student_id, name, email
registrations.json	event_id, student_id, registered_at
feedback.json	event_id, student_id, rating, comment, submitted_at

CHAPTER 5: SYSTEM FLOW

The diagrams below show who uses the system, how data moves through it and how the registration flow behaves. Each figure is followed by a short description.

Context Level Diagram (Fig 5.1): The single system "Campus Event Management & Feedback Analyzer" interacts with two external entities. The **Administrator/Organiser** sends event details and receives event lists, participation summaries and analysis reports. The **Student** sends registration details and feedback and receives confirmation or error messages.

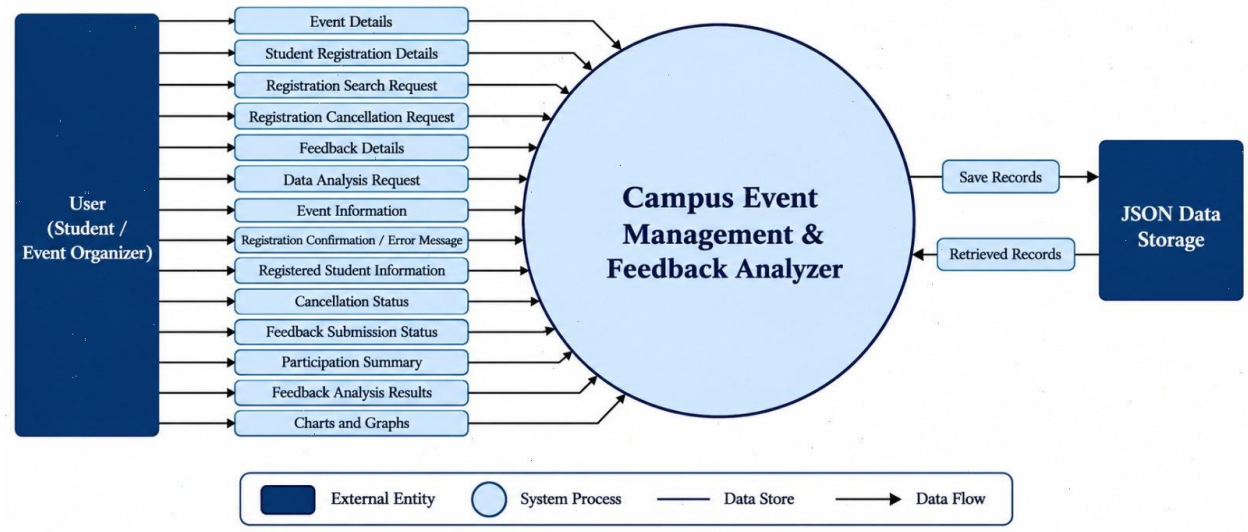


Fig 5.1: Context Level Diagram

Data Flow Diagram Level 0 (Fig 5.2): User inputs (event details, student details, feedback) enter the central process "Event Management System". The process reads from and writes to the JSON data store and returns tables, messages and charts to the user interface.

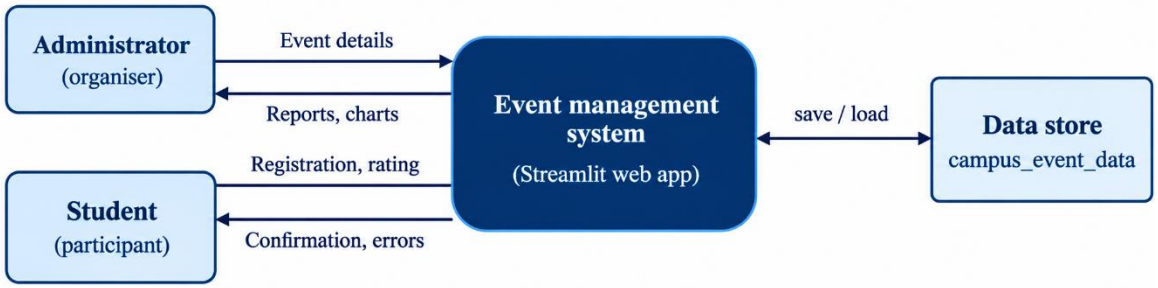


Fig 5.2: Data Flow Diagram Level 0

Data Flow Diagram Level 1 (Fig 5.3): The central process is divided into four sub-processes: (1) Manage Events, which reads and writes events.json; (2) Manage Registrations, which reads events.json and writes students.json and registrations.json; (3) Collect Feedback, which reads events.json and registrations.json and writes feedback.json; and (4) Analyse & Visualise, which reads all four files and produces tables and charts.

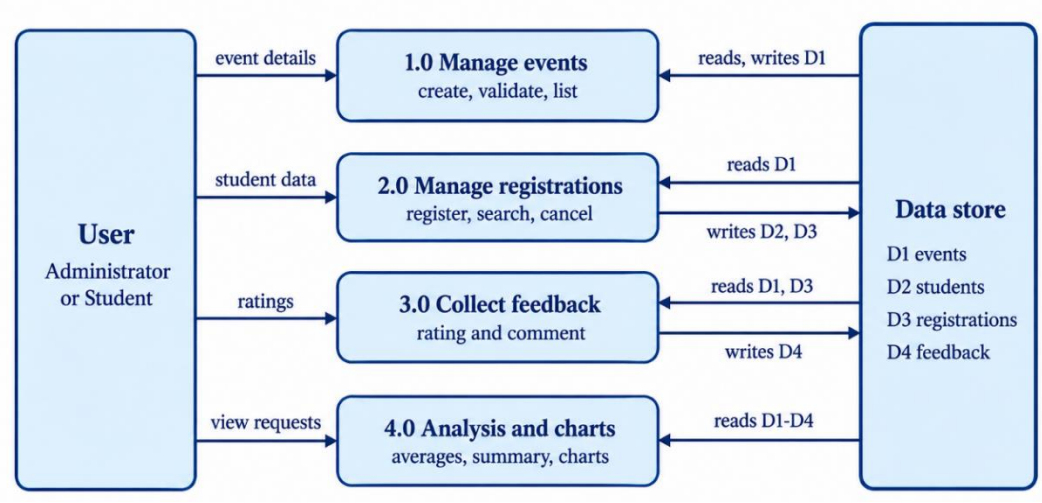


Fig 5.3: Data Flow Diagram Level 1

Use Case Diagram (Fig 5.4): The actor Administrator can create events, view events, view registered students, cancel registrations, view the participation summary, analyse feedback and generate charts. The actor *Student* can register for an event and submit feedback.

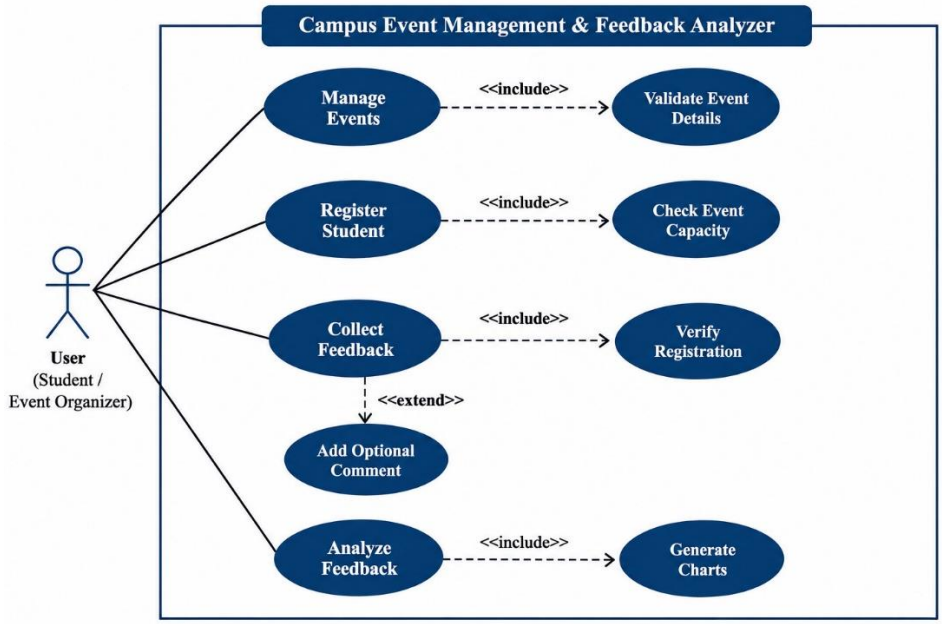


Fig 5.4: Use Case Diagram

Activity Diagram (Fig 5.5): The user selects a menu option → enters data → the system validates it → if validation fails, an error message is shown; if it passes, the record is saved to the JSON file and a success message is shown.

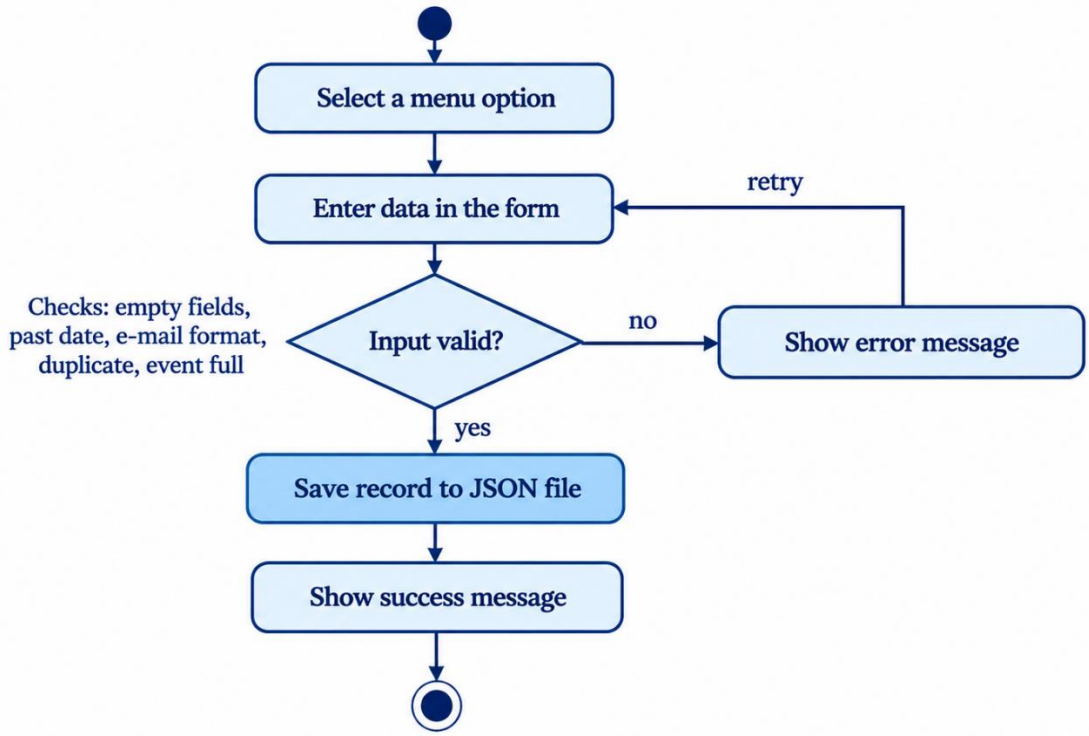


Fig 5.5: Activity Diagram

Sequence Diagram (Fig 5.6) – Student registration: Student → Streamlit UI (submits form) → CampusEventManager (get_event, check duplicate, check capacity) → Student/Registration objects (created) → JSON files (saved) → UI (success or error message).

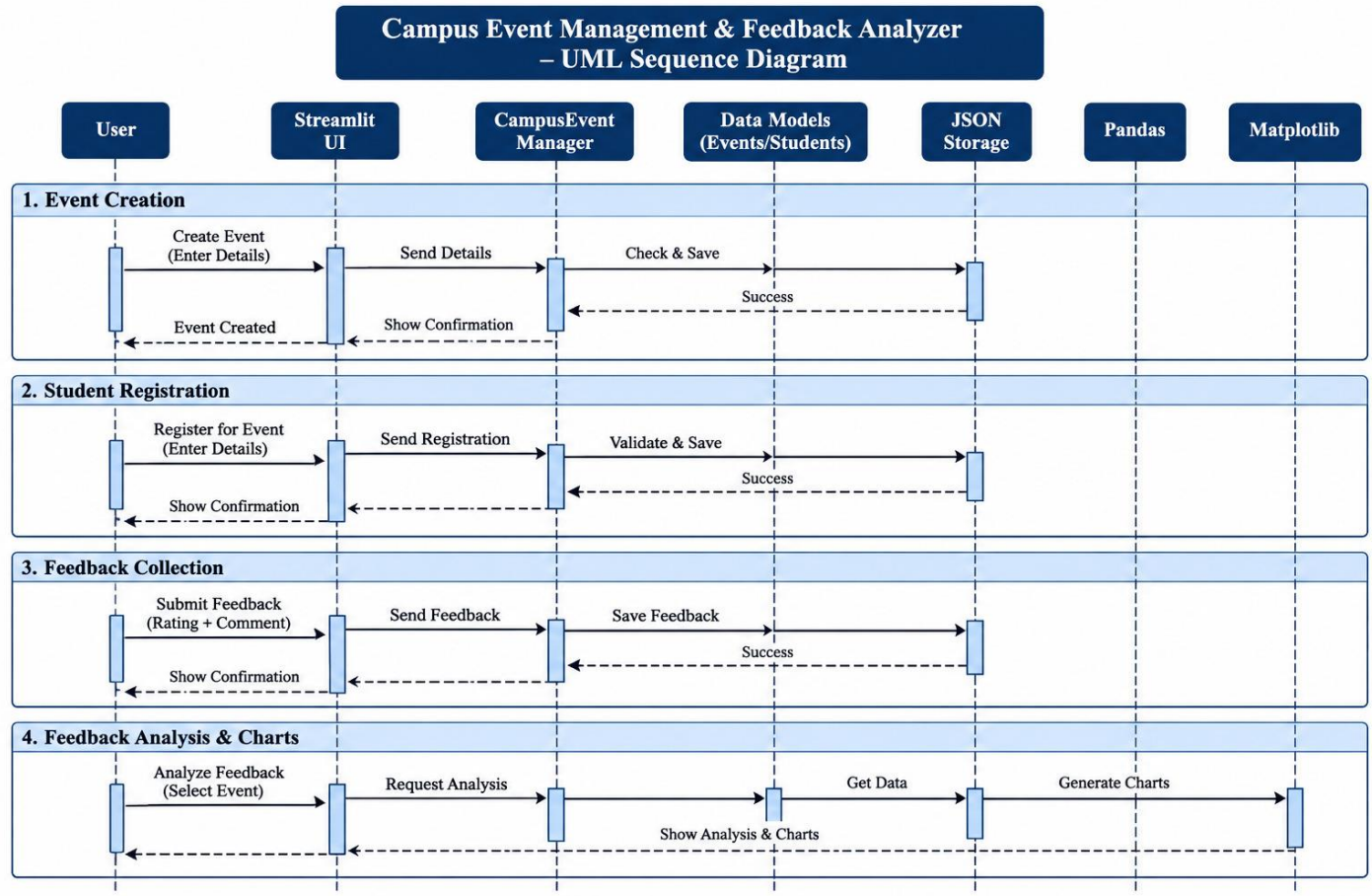


Fig 5.6: Sequence Diagram

Flow Chart (Fig 5.7) – Registration logic:

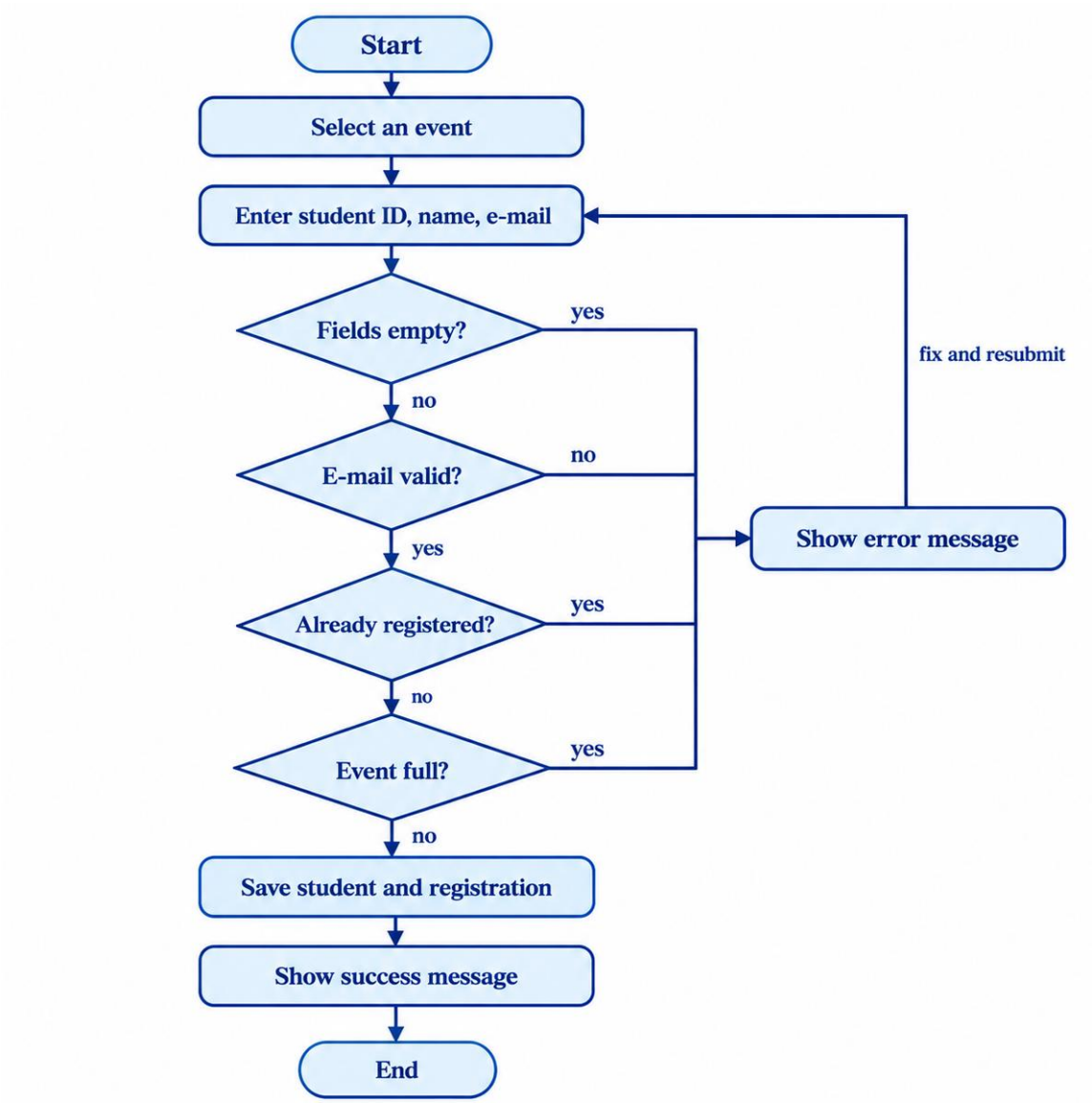


Fig 5.7: Flow Chart

CHAPTER 6: PROPOSED DESIGN

This chapter explains how the application is organised and how each module works. The design follows a layered approach: a data layer (JSON files), a logic layer (classes, validation and exceptions) and a presentation layer (Streamlit pages).

6.1 Data Layer

The constants below define the storage location. The CampusEventManager constructor creates the folder if needed and loads all four files into Python lists.

```
DATA_DIR = "campus_event_data"
EVENTS_FILE = os.path.join(DATA_DIR, "events.json")
STUDENTS_FILE = os.path.join(DATA_DIR, "students.json")
REGISTRATIONS_FILE = os.path.join(DATA_DIR, "registrations.json")
FEEDBACK_FILE = os.path.join(DATA_DIR, "feedback.json")
```

load_data() returns an empty list if a file is missing, unreadable or does not contain a list, so a damaged file never crashes the application. save_data() writes with indentation and UTF-8 encoding, and save_all() persists all four lists.

6.2 Classes

Class	Purpose
Event	Holds event_id, name, date, venue and max_participants. Its constructor re-validates the date format (YYYY-MM-DD) and rejects past dates, so the rule is enforced even if the user interface is bypassed.
Student	Holds student_id, name and e-mail.
Registration	Links an event_id and student_id and stamps the registration time automatically.
CampusEventManager	Loads and saves data, finds an event by ID (get_event) and counts registrations (registered_count).

Each data class has a to_dict() method that converts the object to a dictionary for JSON storage.

6.3 Custom Exceptions

A base class ProjectError is extended by four specific exceptions, which make error handling readable.

Exception	Raised when
InvalidEventIDError	The event ID does not exist.
DuplicateRegistrationError	The student is already registered for the event.
EventFullError	The registered count has reached the maximum.
RegistrationNotFoundError	No matching registration exists (cancellation or feedback).

6.4 Email Validation

The function is_valid_email() applies stricter checks than a basic pattern. It requires exactly one "@", a maximum length of 254 characters, a local part of at most 64 characters built from letters, digits and the symbols . _ % + - (no leading, trailing or consecutive punctuation), at least two domain labels each up to 63 characters without leading or trailing hyphens, and a final alphabetic extension of at least two letters.

```
if not re.fullmatch(r"[A-Za-z0-9]+(?:[._%+-][A-Za-z0-9]+)*", local_part):
    return False
```

6.5 Modules of the Application

1. View All Events: Builds a table from all events and shows the participant count in the form registered/maximum.

2. Create Event: A form collects ID, name, date (date picker), venue and maximum participants (minimum 1). On submission the application checks that the date is not in the past, the ID is not empty and not already used, and the name and venue are not empty. A valid event is created through the Event class and saved.

3. Register Student: The user selects an event and enters the student's ID, name and e-mail. The checks run in this order: required fields, e-mail format, duplicate registration, event capacity. If the student ID is new, a student record is created; a registration with a timestamp is then added and all data is saved.

```
if manager.registered_count(selected_event_id) >= event["max_participants"]:
    raise EventFullError("This event is full. Registration is closed.")
```

4. Search Registered Students: For a selected event, the application joins registrations with student records and shows ID, name, e-mail and registration time.

5. Cancel Registration: The user enters an event ID and a student ID. If the event and the matching registration exist, the registration is removed and the file is saved; otherwise a clear error is shown.

6. Collect Feedback: The user enters the event ID, student ID, a rating from 1 to 5 (slider) and an optional comment. Feedback is accepted only if the event exists, the student is currently registered for it, and the student has not already submitted feedback for that event.

7. Participation Summary: Shows each event with its date, venue, number of participants and maximum participants.

8. Analyze Feedback: Converts feedback into a pandas DataFrame, ensures ratings are numeric, maps event IDs to event names and displays the total number of responses, the overall average rating, and a table of average rating and response count per event.

9. Generate Charts: Uses matplotlib to draw (a) a bar chart of average rating per event (scale 0–5), (b) a histogram of the distribution of ratings from 1 to 5, and (c) a bar chart of participants per event. Charts based on feedback appear only when feedback exists.

6.6 Why This Design Works

- Business rules are checked before any data is written, so stored data stays consistent.
- Errors are shown as readable messages in the interface instead of program crashes.
- Every successful change is saved immediately, so no data is lost when the page is refreshed.
- No database or external service is required, so the project is easy to install and demonstrate.

CHAPTER 7: EXPERIMENTAL RESULT

The results focus on how the CSV data was processed, how event and feedback metrics were calculated, how statistics were produced, and how interactive charts were generated. Screenshots of the Streamlit dashboard,charts,and sample reports can be inserted in ther espective places.

1.Data Loading Stage

The system successfully reads the events. csv, registrations. csv, and students. Csv files and prepares the data for analysis.

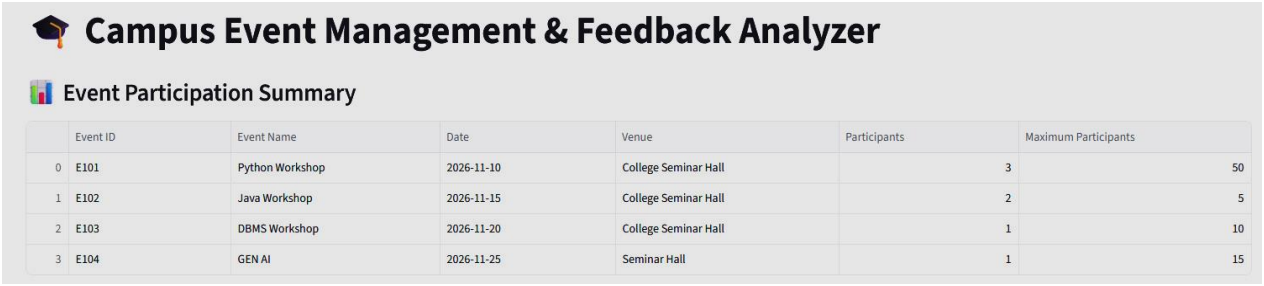


	Event ID	Name	Date	Venue	Participants
0	E101	Python Workshop	2026-11-10	College Seminar Hall	3/50
1	E102	Java Workshop	2026-11-15	College Seminar Hall	2/5
2	E103	DBMS Workshop	2026-11-20	College Seminar Hall	1/10

Fig 7.1: Data Loading Stage

2. Event & Registration Analysis

Registration counts, attendan cerates, and even trankings are calculated for each event and for the campus overall.



	Event ID	Event Name	Date	Venue	Participants	Maximum Participants
0	E101	Python Workshop	2026-11-10	College Seminar Hall	3	50
1	E102	Java Workshop	2026-11-15	College Seminar Hall	2	5
2	E103	DBMS Workshop	2026-11-20	College Seminar Hall	1	10
3	E104	GEN AI	2026-11-25	Seminar Hall	1	15

Fig 7.2: Event Statistics

3.Feedback Analysis

Average ratings are computed, rating distributions are generated, and events falling below a quality threshold are identified.



Fig 7.3: Registration/ Feedback Analysis

4.Charts Generation

Interactive Plotly charts(bar,pie,histogram,ranking) are produced for registration counts,rating distributions, andperformance comparison.

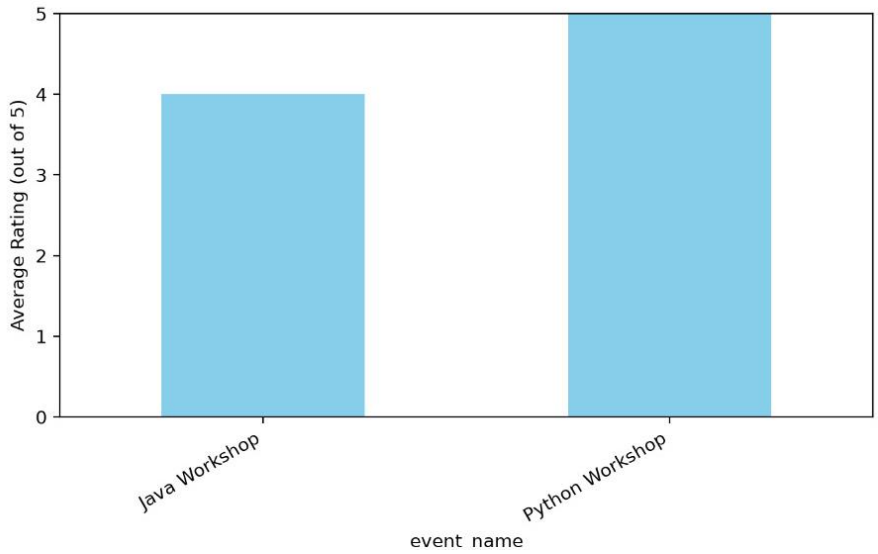


Fig 7.4: Average Feedback Rating by event

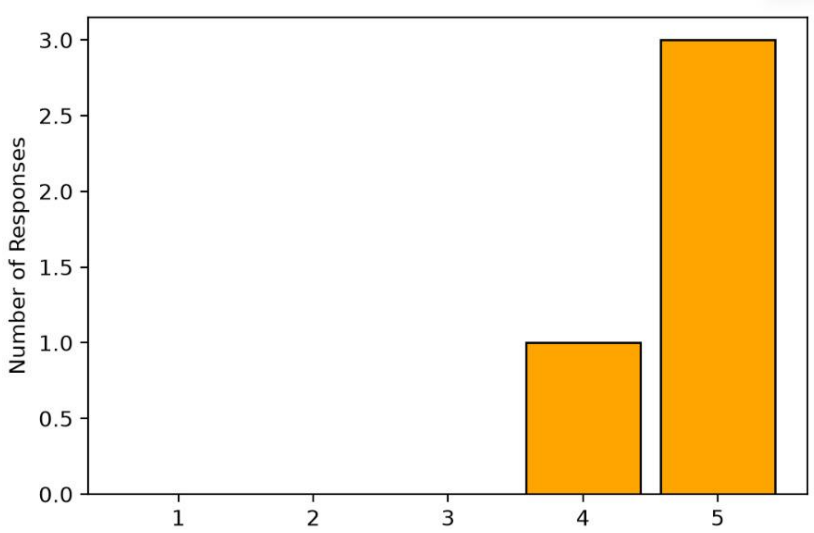


Fig: 7.5: Distribution of Feedback Ratings

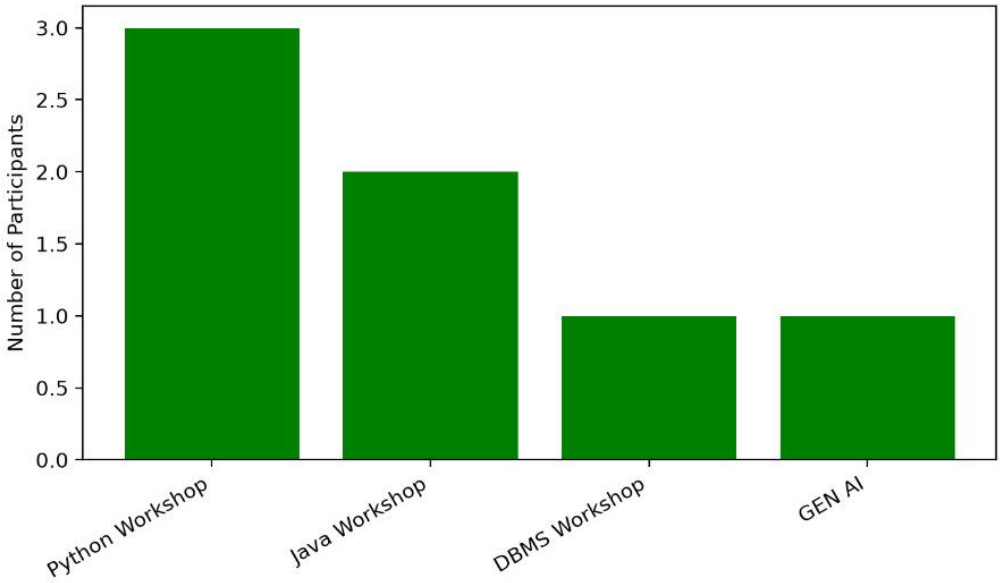


Fig: 7.6: Event-wise Participation

CHAPTER 8: FUTURE SCOPE

Several improvements can be made in later versions:

1. **Authentication and roles:** Separate login for administrators and students so that only administrators can create events or cancel registrations, and students can only act on their own records.
2. **Database integration:** Replace JSON files with SQLite, MySQL or PostgreSQL for multi-user access, faster searching and stronger integrity.
3. **Notifications:** Send confirmation and reminder e-mails or messages to registered students.
4. **Feedback timing and rules:** Allow feedback only after the event date and optionally keep or remove feedback when a registration is cancelled.
5. **Text analysis of comments:** Apply sentiment analysis to the comment field to summarise common praise and complaints.
6. **Reports and export:** Export participant lists, summaries and charts as CSV, Excel or PDF.
7. **Waiting list and attendance:** Add a waiting list for full events and QR-code based attendance marking.
8. **Event editing and categories:** Support editing or deleting events, and categorising events (technical, cultural, sports) for category-wise analysis.

CHAPTER 9: CONCLUSION

The project successfully delivers a complete, working system for managing campus events and analysing feedback. It covers the whole life cycle of an event: creation, student registration, participant tracking, cancellation, feedback collection and analysis. Strict validation of dates, e-mail addresses, capacity and duplicate entries keeps the stored data reliable, while custom exceptions and clear messages make the application easy to use.

By combining Python, Streamlit, pandas and matplotlib with simple JSON storage, the project shows that a practical, data-driven application can be built with modest tools. The graphical analysis gives organisers a clear picture of participation and satisfaction, helping them plan better events. Overall, the project meets its objectives and provides a solid base for future extension with authentication, databases and richer analytics.

REFERENCES

1. Streamlit Inc. *Streamlit Documentation*. Available at: https://docs.streamlit.io
2. The pandas development team. *pandas Documentation*. Available at: https://pandas.pydata.org/docs/
3. Hunter, J. D. (2007) *Matplotlib: A 2D Graphics Environment*, Computing in Science & Engineering. Documentation available at: https://matplotlib.org/stable/
4. Python Software Foundation. *Python 3 Documentation – json, re and datetime modules*. Available at: https://docs.python.org/3/